

Project Task List 3 - Technical Design & REST API Implementation

GitHub Link : <https://classroom.github.com/a/EWZ7aGi6> (<https://classroom.github.com/a/EWZ7aGi6>)

Tech Stack

- Next.js (TypeScript, App Router)
- PostgreSQL (Neon)
- Prisma ORM
- Tailwind CSS + shadcn/ui
- REST API (Route Handlers)
- Zod (validation)
- Better-Auth (<https://www.better-auth.com/>) for Authentication & Role-Based Access

Each team works only on **their assigned module**, but it must integrate with the full College Scheduling System.

Task 1: Database Design (Prisma)

Deliverable:

Your module's schema.prisma models.

Requirements:

1. Define all tables related to your module.
2. Use proper Prisma attributes:

- @id
- @default(uuid())
- @unique
- @relation

1. Include:

- Enums (for roles, status, types)
- Timestamps (createdAt, updatedAt)

1. Define relationships with other modules.

2. Add at least:

- One index
- One constraint (e.g., unique combination)

Task 2: UI Implementation Plan

(No rough sketches. Plan real UI structure.)

Deliverable:

- UI Architecture Document
- Fully implemented static UI pages (3–5 screens)

Requirements:

1. List the main screens (3–5 screens).
2. For each screen, describe:
 - Page layout (grid/sections)
 - Components used (from shadcn/ui)
 - Buttons, forms, tables, filters

1. Clearly label:
- Which components are "use client"

• Which are Server Components
1. Explain how data is fetched from your API.
2. Fully implemented static UI pages

Task 3: REST API Design

Your module must expose REST endpoints so other dashboards can integrate.

Deliverable:

API Specification (simple table format)

Requirements:

1

Define CRUD Endpoints

Example format:

Method	Endpoint	Purpose
GET	/api/rooms	Get all rooms
GET	/api/rooms/:id	Get room
POST	/api/rooms	Create room
PUT	/api/rooms/:id	Update room
DELETE	/api/rooms/:id	Delete room

2 Validation

- Use Zod
- Reject invalid data
- Return proper HTTP status codes (400, 404, 500, etc.)

3 Type Safety

Define and export TypeScript interfaces for:

- Input data
- API response data

No any allowed.

Task 4: Data Flow Explanation

Deliverable:

Simple flow diagram showing:

User → UI Form → REST API → Prisma → Neon DB → Response → UI Update

Also show what happens if:

- Validation fails
- Database error occurs

Task 5: Security

You must define:

- User roles
- Who can access which endpoints
- Basic role-based access rules

Final Submission Must Include

1. Prisma schema (your module only)
2. API endpoint list
3. TypeScript interfaces
4. UI structure document
5. Fully implemented static UI pages
6. Data flow diagram
7. Role access explanation